

Hardware-Conscious Performance Engineering of Fainder for Distribution-Aware Dataset Search

Master's Thesis Defense

Tarik Abu Mukh

August 8, 2026

Technische Universität Berlin

Chair of Database Systems and Information Management (DIMA)

Reviewers: Prof. Dr. Volker Markl, Prof. Dr. Matthias Böhm Advisor: Lennart Behme

Problem

Distribution-Aware Dataset Search and Fainder

- Setting: search over millions of datasets where raw data is not accessible (privacy, licensing) — only per-column **histograms** are published.
- Predicate class: *“find all columns where the p -th percentile exceeds threshold t .”*
- **Fainder** [Behme et al., VLDB 2024]: an index for this problem.
 - K-means clusters histograms, aligns each cluster onto a shared bin grid.
 - Precomputed cumulative densities $\Rightarrow$ percentile predicates become **binary search**.
 - On GitTables (5M histograms): >2 orders of magnitude over profile scan, index shrinks from >1 TB to 3.2 GB.
- This thesis: how do we execute Fainder’s *query phase* efficiently on a modern multi-core server?

The Problem: Python Runs the Interpreter, Not the Algorithm

- Reference implementation: Python + NumPy.
- **GIL** serialises the per-cluster loop — wall time is *flat* across all thread counts (2% spread).
- AoS layout pollutes cache lines with unread hist-id fields.
- Per-iteration interpreter dispatch compounds onto both.

10,000 queries, 5M histograms
~2.4 hours

(8,589 s, any thread count)

- Interactive exploration untenable.

- Curious detail: Python's IPC is *higher* than Rust's (2.35 vs 0.96) — it retires $6.2\times$ the instructions on bookkeeping. The win is fewer instructions on more cores, not better IPC.

Goal, Scope, Approach

- **Goal:** close the gap between Fainder's algorithmic potential and what its implementation achieves — without changing results.
- **Approach:** re-implement the query phase in **Rust** (PyO3 extension, same Python API), then run an ablation study across >20 hardware-conscious optimisation axes.
- **Hard constraint:** bitwise-identical results to the reference on all workloads (precision = recall = 1.000).
- **Out of scope:** algorithm changes, GPU, distributed execution.

The core of the thesis is the investigation, not the port

Four binding hardware ceilings, a negative-composability pattern across five axes, and a pre-flight methodology that follows from it.

Method

Experimental Setup

Hardware (representative of a search-service deployment node):

- 2× Intel Xeon Platinum 8468H (Sapphire Rapids), 96 physical / 192 logical cores.
- 105 MiB L3 per socket, 1 TB DDR5-4800, UPI cross-socket ($\sim 2.1\times$ remote-load penalty).

Workloads (GitTables, 10K percentile queries shared across all):

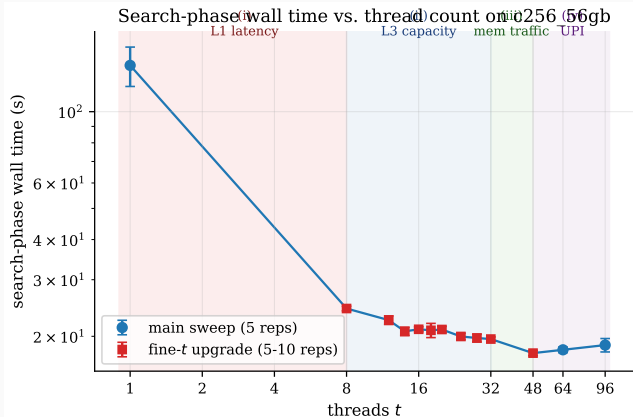
Dataset	n_{hists}	K_{eff}	hists/cluster	role
c256_10gb	324 K	129	~ 2.5 K	small
c256_30gb	997 K	184	~ 5.4 K	medium
c256_56gb	5.02 M	191	~ 26 K	primary
c1024_56gb	5.02 M	610	~ 8.2 K	out-of-distribution

Protocol: 5-rep medians, thread sweep $t \in \{1 \dots 96\}$, perf counters per cell, warm cache, suppress-results vs with-results separated. All claims trace to a benchmark database (2,658 runs).

- Query engine as a compiled extension module (PyO3 + Maturin); clustering and loading stay in Python.
- Rayon work-stealing parallelism over clusters; no GIL in the hot path.
- Optimisation axes as Cargo feature flags, so every variant is a separate measurable build:
 - **Layout:** SoA vs AoS, f16 half-precision, Eytzinger, packed IDs, per-cluster reindexing.
 - **Search:** AVX-512 SIMD leaf stage, 8-way batch, horizontal lockstep, PGM learned index.
 - **Scheduling:** column-centric engine, query-batch, morsel-driven scheduler, core pinning.
 - **Memory:** pooled output buffers, mimalloc, NUMA placement.
- Rebinning (index construction) also ported: one fused parallel kernel per cluster.
- Every build validated bitwise-identical to the Python reference.

Results: Four Ceilings

Finding 1: Four Performance Ceilings



1. **Compute / L1 latency** ($t \leq 8$)
— candidate, **refuted**.
2. **Shared L3 capacity** ($t \approx 16$).
3. **On-socket memory-traffic pressure** ($t \in [32, 48]$).
4. **Cross-socket UPI** ($t > 48$).

Each binds in a different regime; each responds to a different class of interventions.

Ceiling (i): The Literature's Default — Refuted Here

- Binary search is a serial dependent-load chain; the default assumption is that load latency is the wall.
- Four latency-side ablations, all **null or marginal** ($0.90\text{--}1.13\times$ across all cells):
 - AVX-512 leaf-stage SIMD
 - 8-way pipelined batch-search
 - 16-query horizontal-SIMD lockstep
 - PGM learned index
- Why: the bin-boundary array (~ 142 entries) sits in two L1d lines; the column slice is L2-resident. The chain is short and cache-friendly — there is no latency headroom to recover.

A refutation is a finding

On this workload and hardware, no latency-side intervention moves the wall. The action is all above L1.

Ceiling (ii): Shared L3 — Two Mechanisms, Not One

- pooled (pre-allocated output buffers): **-46.9%** at $t=1$, **-34.1%** at $t=8$, **-20.2%** at $t=16$ on the primary workload.
 - Binding mechanism at moderate t : **glibc allocator-arena mutex**, not raw L3 capacity.
- f16 (halved footprint): **no standalone win** on the primary workload; inside the bundle it is a **+15.6% drag** at $t=16$.
 - Same feature on the sparser OOD workload: **saves 20.9%** inside the bundle.
 - Sign flips with cluster density — footprint halving only pays when it unlocks a new cache-resident regime.
- AoS negative control: **+17%** penalty at $t=8$, flips to **-4%** at $t=64$ — the binding ceiling itself moves with t .

Ceiling (ii) = arena-mutex serialisation on dense clusters, aggregate L3 footprint on sparse ones.

Ceilings (iii) and (iv): Memory Traffic and the Socket Boundary

Ceiling (iii): on-socket memory-traffic pressure ($t \in [32, 48]$)

- Anti-scaling: wall time *rises* as cores are added.
- Not “DRAM bandwidth”: measured traffic is a small fraction of STREAM peak. Pressure, not saturation.
- Interventions that reduce bytes per comparison / per emitted result apply here.

Ceiling (iv): cross-socket UPI ($t > 48$)

- Separated from (iii) by a four-configuration numactl probe.
- Single-socket placement saves 11–13% at $t \leq 48$.
- Page-granular interleave across sockets **regresses +26–41%**: 4 KB round-robin defeats the prefetcher.

Results: Negative Composability

Finding 2: Optimisations Compose Negatively

Five ablation pairs, five different hardware axes, one shape: *the coarser optimisation already absorbed the headroom; the finer one's overhead becomes net cost.*

Composition	Intended axis	Outcome
best-combo + packed-ids	emit-ID bandwidth	NEG 12/18 cells, up to +24%
best-combo + qbatch $K=128$	cold-load amortisation	NEG every cell, +8 to +32%
qbatch + morsel	cross-worker coordination	NEG every cell, +8 to +12%
packed-ids + local-ids	read-side bandwidth	NEG +186 to +245%
first-touch + interleave	memory-locality balance	NEG every cell, +26 to +41%

- Each pair had a plausible argument for positive composition. Each was falsified by perf counters.
- Cross-axis breadth rules out per-axis coincidence: the pattern is **structural**.

Finding 3: Pre-Flight Ceiling Identification

- Classical ablation methodology: implement, measure against default, accept if faster.
- That assumes an ablation's axis is independently informative. With stacked ceilings, it is not: whether an optimisation composes depends on the binding ceiling *at the target composition*, not at the default build.

The pre-flight check, before writing code:

1. Probe the binding ceiling at the *target* composition with perf counters.
2. Verify the ablation's axis still binds there.
3. If it does, predict the saving via bandwidth-budget arithmetic.
4. Measure across the regime cells where the ceiling binds; decompose saving vs bookkeeping cost.
5. A negative composition is a result, not a failure.

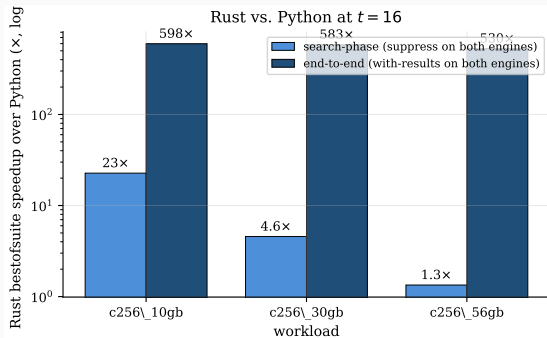
Applies to any hardware-near system bound by multiple stacked ceilings — not Fainer-specific.

Dispatch-on-Regime: The Deployment Artefact

- Negative composability $\Rightarrow$ no single build dominates. The only near-optimal deployment is a **per-regime build selection**.
- Input: $(n_{\text{hist}}, n_{\text{threads}})$. Output: build string + environment flags.
 - `best-combo` (= pooled f16 simd pin-cores cluster-prefetch mimalloc) at moderate t on big data; `packed-ids` at $t=64$; `query-batch` at $t=96$; `column-centric` at low t ; default on small data.
- **Validation:**
 - In-distribution: picks the regime-best build in 17/18 cells exactly, 18/18 within 5%.
 - Out-of-distribution (c1024_56gb, 3.2 $\times$ sparser clusters): 6/6 within 5%, all within 1%.
 - Combined: 23/24.
- Regime boundaries are workload- and machine-specific; the probing procedure behind them is not.

Results: vs Python

The Honest Speedup Story



Workload	search	end-to-end
c256_10gb	22.7×	598×
c256_30gb	4.6×	583×
c256_56gb	1.34×	530×

- End-to-end: 2.4 h $\rightarrow$ 16 s on the primary workload.
- $\sim 99\%$ of that ratio is avoiding Python's per-(query, cluster) result-dict construction — **not** search superiority.
- Search phase bends toward the shared hardware ceilings as footprint grows.

Bonus: the Rust rebinning kernel cuts index-construction alignment by $4.7\times$ (small/medium) and $2.2\times$ (5M histograms).

Conclusion

Contributions

1. **Rust re-implementation** of Fainder's query pipeline — bitwise-identical results, 530–598× end-to-end, integrated via PyO3. *engineering*
2. **Four-ceiling characterisation** of distribution-aware search on Sapphire Rapids, established via perf counters. *scientific*
3. **Negative-composability pattern**: five independent instances on five axes. *scientific*
4. **Pre-flight ceiling identification**: probe the binding ceiling at the target composition before paying implementation cost. *methodological*
5. **Dispatch-on-regime policy**, validated 23/24 within 5%, including out-of-distribution. *engineering*

(1) and (5) are the staging ground; (2)–(4) are what generalises.

- The speedups are specific to this codebase, micro-architecture, and workload.
- The structural finding travels: on hardware-near workloads, whether optimisation B helps depends on which ceiling optimisation A has already collected — invisible to per-operator cost models.

One sentence

Identify the binding ceiling at the target composition *before* committing the implementation cost of the next optimisation.

Questions?

Backup

Backup: Why is end-to-end $530\times$ if search is only $1.34\times$?

- Python's per-(query, cluster) result-dict construction dominates its wall. Measured at $t=16$, materialisation accounts for:
 - c256_10gb: 580.9 s of 599.7 s (96.9%)
 - c256_30gb: 1,878.5 s of 1,890.3 s (99.4%)
 - c256_56gb: 8,568 s of 8,589 s (99.8%)
- Rust emits results without per-cluster dict churn, so it dodges this cost entirely.
- That is why we report both columns: suppress-vs-suppress isolates the search engines; with-results-vs-with-results is what a user experiences.
- Both numbers are honest; conflating them would not be.

Backup: Why doesn't f16 win standalone?

- Prediction: halve the footprint, win in the regime where f32 exceeds L3 but f16 fits.
- On c256_56gb the per-cluster f32 slice (~ 105 KiB) is already L2-resident. Halving it unlocks no new cache-resident regime, but every comparison pays the f16 $\rightarrow$ f32 dequantisation.
- Result: no significant standalone win in the 10-rep fine- t sweep; a +15.6% drag inside the bundle at $t=16$.
- On c1024_56gb ($3.2\times$ sparser, ~ 33 KiB slices, 610 clusters) the aggregate cluster-loop footprint is the constraint — there f16 carries $\sim 20\%$ of the bundle win.
- General point: footprint optimisations are density-conditional, not universal.

Backup: Why “memory-traffic pressure” and not “DRAM bandwidth”?

- STREAM Triad on this machine reaches ~ 330 GB/s aggregate.
- Accounting the workload’s LLC-miss traffic (64 B per missed probe) puts it at $\sim 0.5\%$ of that peak.
- “Bandwidth-bound” would claim saturation of the DRAM channels; the counters show nothing close.
- What we observe is anti-scaling from aggregate pressure on the on-socket memory subsystem (queues, MLC occupancy, allocator traffic) — hence *on-socket memory-traffic pressure*.
- The naming matters: it predicts which interventions help (fewer bytes per comparison/emit), and that pure bandwidth upgrades would not.

Backup: The local-ids regression — mechanism

- `local-ids` (per-cluster reindexing) was pre-registered as a positive prediction: narrower IDs, less read-side bandwidth.
- Measured: **+186 to +245%** at $t \in [8, 64]$ on top of `packed-ids` — our falsified prediction.
- Mechanism, from a direct `perf stat` probe: the $\text{bw} \approx 13$ decoder's access pattern over the packed buffer drives the **dTLB miss rate up** $52\times$ per instruction.
- TLB-walk stalls (~ 100 cycles each) drown the bandwidth saving.
- Future work (F1): Transparent Huge Pages (2 MiB pages) would cut TLB-miss frequency $512\times$ per covered page — the natural follow-up probe.

Backup: Does any of this generalise beyond one machine?

- Honest answer: the *thresholds* do not, the *methodology* does.
- All experiments ran on one Sapphire Rapids 8468H configuration. Genoa, Granite Rapids, or Grace would shift per-ceiling thresholds and dispatch boundaries.
- What transfers: the pre-flight procedure (probe the binding ceiling at the target composition) is machine-agnostic; the four-ceiling taxonomy is a hypothesis set to re-test, not a constant.
- Density scope: dispatch is validated between $\sim 8,200$ and $\sim 26,300$ hists/cluster; outside that range the policy is a prediction.
- Multi-machine deployment would add a fifth candidate ceiling (network), with its own intervention class.

Backup: How was correctness validated?

- Every Rust build validated **bitwise-identical** to the Python reference across all workloads: precision = recall = 1.000.
- Validation runs in the benchmark harness itself, not as a one-off test — each measured configuration is also a correctness check.
- Rebinning kernel: verified against the Python reference on 200 queries. Rust rounds half-away, NumPy uses banker's rounding; maximum cumulative difference 10^{-4} at a handful of rows, no effect on sort order or query results.
- Kernel falls back to the Python pipeline if the extension is not importable or for `cubic_spline` bin estimation.

Backup: Why Rust, and not C++, Cython, or Numba?

- The requirement set: no GIL in the hot path, explicit control over layout and allocation, safe shared-memory parallelism, and a maintainable Python integration.
- Rust + PyO3 gives all four: Rayon work-stealing without data races, Cargo feature flags to make every ablation a separate reproducible build, and a drop-in extension module for the existing API.
- C++ could reach the same performance; the feature-flag ablation matrix and memory safety under 96-thread parallelism are where Rust paid off in practice.
- Numba/Cython remove less of the interpreter overhead and keep the allocator and layout decisions in Python's hands — the layers our measurements show are the actual cost.

Backup: Why does the rebinning speedup drop at 5M histograms?

- $4.72\times$ (324 K) and $4.77\times$ (997 K) drop to $2.18\times$ at 5.02 M histograms.
- Cause: the kernel's own parallelism ceiling — at that scale it runs at ~ 2 effective cores of 96, with $\sim 46\%$ of kernel time on a shared per-cluster allocation.
- Not caused by “NumPy bulk operations dominating” — that earlier hypothesis was tested and refuted.
- Construction-time cost, paid once per index build; the query-phase findings are unaffected.